

FREE RESOURCE

BUILD \$5K WEBSITES USING CLAUDE CODE

The exact workflow behind a scroll-driven luxury product landing page: Claude Code + Higgsfield MCP + Next.js, step by step.

By **Daksham Bugreja** | `@dak.bugreja`

I. WHAT YOU'RE BUILDING

This guide walks you through the exact workflow behind the video: a scroll-driven luxury product landing page where the video is controlled by the user's scroll position, frame by frame.

By the end you'll have a full production-ready Next.js site with a canvas-driven video hero, a features section, a specs table, and a closing CTA. All of it built by Claude Code using a single prompt. No web development background needed.

Why \$5K? A scroll-driven product landing page like this, built by a traditional developer or agency, runs \$4,000 to \$12,000. This workflow produces the same output in one Claude Code session. The client never needs to know how long it took.

II.

WHAT YOU NEED

Before you start, have these set up:

- **Claude Code:** Claude Pro subscription (\$20/month). Your AI coding environment. Comment SETUP if you need the install guide.
- **Higgsfield account:** The AI video and image platform that generates your assets. Free tier available to start.
- **Higgsfield API key:** Found in your Higgsfield dashboard under Settings. You'll connect this to Claude Code via MCP.
- **Node.js 18+ and npm:** Required to run the Next.js project Claude scaffolds. Download from nodejs.org if you don't have it.
- **The prompt:** Section V of this guide. Paste it into Claude Code exactly as written.

III.

STEP-BY-STEP SETUP

This is the exact sequence from the video. Do these in order. Skipping a step is the most common reason it doesn't work.

01 Open Claude Code and go to Connectors

In the Claude Code sidebar, find the Connectors section. This is where you add MCP servers that extend what Claude can build.

02 Add a Custom Connector for Higgsfield

Click "Add Custom Connector". Name it "Higgsfield". Enter the MCP server URL and paste your Higgsfield API key when prompted. Save the connector.

03 Open a new Claude Code session

Start a fresh Claude Code chat. Confirm that the Higgsfield connector is active. You'll see it listed under active tools.

04 Paste the prompt from Section V

Copy the full prompt exactly as written and paste it into your Claude Code session. Do not modify it on the first run. See how it performs, then adapt it to your client's niche.

05 Let Claude run all 8 steps

Claude generates Higgsfield assets, extracts video frames, scaffolds the Next.js project, and builds every component. The full run takes 5 to 10 minutes. Do not interrupt it.

WHAT GETS GENERATED

One prompt. Here is everything Claude and Higgsfield produce automatically by the time it finishes:

HIGGSFIELD ASSETS

- **Studio product image:** A high-end product photograph generated by Higgsfield. Pure black background, camera-ready quality.
- **Exploded view image:** Every component of the product deconstructed and floating in radial symmetry. Used as the second reference for the hero video.
- **Hero video:** A cinematic orbit that transitions mid-shot from the assembled product into a full mechanical deconstruction. Exported as hero.mp4.

CODE AND PROJECT FILES

- **Extracted JPEG frames:** Every frame of the video pulled out by ffmpeg and placed in the public directory. This is what the canvas scrubs through on scroll.
- **Full Next.js project:** Scaffolded with Next.js 15, React 19, Framer Motion, and TypeScript. Zero configuration needed.
- **ScrollHero component:** Canvas-driven frame scrubber. Scroll controls the video frame by frame using a requestAnimationFrame loop. No scroll event listeners.
- **FeaturesSection, SpecsSection, ClosingCTA:** Three production-quality page sections with real luxury copy, Framer Motion scroll animations, and accurate product specs.

Pro tip: The prompt works for any product with a pure black background video. Swap the watch details for your client's product and the copy writes itself. The more specific your Higgsfield image prompts, the more premium the output.

V.

THE PROMPT

Paste this into Claude Code with your Higgsfield connector active. Run it exactly as written on the first session, then customise the product details, copy, and specs for your client.

>_ Build a scroll-driven product landing page for a luxury watch using a Higgsfield AI-generated video.

STEP 1 – CREATE THE ASSETS IN HIGGSFIELD AI

Generate these three assets in order:

ASSET 1 – Base product image

Prompt:

"A studio-grade product photograph of a Rolex Day-Date 40 in Everose gold, shown at a three-quarter angle revealing both the dial and the bracelet. Pure black background with zero ambient light bleed, no reflections on the case or crystal, no surface shadows. The watch is fully assembled, bracelet intact. Shot as if for a high-end print campaign – clinical precision, no stylization."

ASSET 2 – Exploded reference image (use Asset 1 as reference)

Prompt:

"Using the provided reference watch image: deconstruct the watch into a precise exploded-view diagram. Each component – bezel, crystal, dial, movement plate, rotor, case, caseback, crown, bracelet links and clasp – should float apart from its assembled position along its natural mechanical axis, with uniform spacing between parts. The arrangement should feel deliberate and symmetrical, like a technical illustration or a luxury brand's campaign visual. Pure black background. All parts retain their Everose gold finish and material texture. No labels, no lines, no graphic overlays."

ASSET 3 – Hero video (use Assets 1 and 2 as references)

Prompt:

"A Rolex Day-Date 40 in Everose gold with a chocolate sunburst dial floats in a pure black void, fully assembled, with no environment, no ground plane, no ambient reflections. The camera begins at a front-right angle and slowly orbits clockwise in a smooth, uninterrupted arc. As the camera arrives at the direct front-facing position – approximately halfway through the shot – the watch begins a seamless mechanical deconstruction. Each component separates along its natural axis with deliberate, weighted momentum: the bezel lifts, the crystal rises, the dial peels back, the movement floats up, the bracelet unfurls and spreads. Parts float outward in perfect radial

symmetry as if gravity has been selectively reversed. The movement is slow, cinematic, and precise – never chaotic. By the end of the shot, all parts are suspended in a balanced exploded arrangement, still against the black void.”

Save the video as hero.mp4. All assets must have a pure black background so they blend seamlessly with the page.

STEP 2 – EXTRACT VIDEO FRAMES

Run ffprobe on the video to get duration, frame rate, and total frames. Then extract all frames as JPEG at 1920px wide:

```
ffmpeg -i hero.mp4 -vf "fps=24,scale=1920:-1" -q:v 3 \
    "frames/frame_%04d.jpg"
```

Count the output frames and store the number as FRAME_COUNT.
Copy the entire frames/ directory to public/frames/.
Also copy hero.mp4 to public/hero.mp4.

STEP 3 – SCAFFOLD THE NEXT.JS PROJECT

Set up package.json with: next 15, react 19, react-dom 19, framer-motion, typescript, @types/react, @types/node.

Add tsconfig.json (standard Next.js App Router config).
Add next.config.ts (empty default export).

Create app/globals.css:

```
* { box-sizing: border-box; margin: 0; padding: 0; }
body { background: #000; color: #fff;
      -webkit-font-smoothing: antialiased; overflow-x: hidden; }
```

Create app/layout.tsx:

- Import Inter and Playfair_Display from next/font/google
- Expose them as CSS variables: --font-inter, --font-playfair
- Set <title> to the watch name

STEP 4 – DESIGN TOKENS

Use these values everywhere. Never deviate:

```
Background:      #000000
Text primary:    #ffffff
Text body:       #E5E5E5
Accent (gold):   #C8A96E    ← Everose gold
Accent hover:    #E8C98E
Dim text:        #888888    ← never below this on black
Border subtle:   rgba(200,169,110,0.2)

Display font:    var(--font-playfair)  – Playfair Display, weight 400
Body font:       var(--font-inter)     – Inter, weight 300-500
Label style:     Inter, 0.65rem, letter-spacing 0.25em, uppercase,
                 color: accent
```

STEP 5 – BUILD `app/components/ScrollHero.tsx`

'use client'. No video element. Use canvas + preloaded JPEG frames.

Structure:

- Outer div: height 300vh, position relative, ref: containerRef
- Inner div: position sticky, top 0, width 100vw, height 100vh, overflow hidden, background #000
- <canvas> inside: display block, width/height 100%, ref: canvasRef
- Overlay div: position absolute, inset 0, flex column, justify-content flex-end

useEffect:

1. Get canvas context. Apply devicePixelRatio scaling.
2. Cover-fit draw function:
 scale = Math.max(cw/iw, ch/ih)
 center the image, fill black behind it
3. Preload all FRAME_COUNT images from /frames/frame_XXXX.jpg (4-digit zero-padded). Draw frame 0 on first image load.
4. Start a requestAnimationFrame loop – NO scroll event listener:
 top = containerRef.getBoundingClientRect().top
 progress = Math.max(0, Math.min(1,
 -top / (container.offsetHeight - window.innerHeight)))
 target = Math.round(progress * (FRAME_COUNT - 1))
 if target !== currentIdx: draw(target)
5. Resize handler: re-size canvas, redraw current frame.
6. Cleanup: cancelAnimationFrame, remove resize listener.

Overlay content (all fade in with Framer Motion, staggered from 800ms delay, ease easeOut):

- Small label: "Est. 1905 · Geneva" in accent color
- h1: Watch name, Playfair Display, clamp(2.4rem,6vw,5.5rem)
- p: One-sentence brand line, Inter 300, #E5E5E5, max 460px
- <a> button: "Explore Collection", background accent, color #000, Inter 500, 0.7rem, letter-spacing 0.18em, uppercase, padding 0.9rem 2.6rem, pointer-events auto

Gradient behind overlay:

```
linear-gradient(to top, rgba(0,0,0,0.88) 0%,  
                rgba(0,0,0,0.35) 50%,  
                transparent 100%)
```

STEP 6 – BUILD THE REST OF THE PAGE

Background stays #000000 throughout. No navbar, footer, or cookie banner. All sections use Framer Motion useInView scroll-triggered fade-ins (y: 24→0, opacity: 0→1, stagger 100ms, ease [0.25,0,0,1], once: true).

— FEATURES – app/components/FeaturesSection.tsx

- Section label: "Crafted Without Compromise" (accent, uppercase)
- h2: Playfair Display, clamp(2rem,4vw,3.5rem)
- 3-column responsive grid (auto-fill, minmax 300px)
- 6 feature cards, each: SVG icon (stroke accent) + label + 1-sentence copy
- Card top border: 1px solid rgba(200,169,110,0.2)
- Write real luxury watch copy. No lorem ipsum.
- Suggested features: Perpetual Movement, Everose Gold, Oyster Waterproofing, President Bracelet, Superlative Chronometer, Cyclops Date

— SPECS – app/components/SpecsSection.tsx

- Section label: "Technical Specifications" (accent, uppercase)
- h2: "The architecture of precision."
- 2-column responsive table (label left in accent, value right in #E5E5E5)
- 8-10 rows: Reference No., Case Diameter, Case Material, Movement, Power Reserve, Accuracy, Crystal, Water Resistance, Dial, Bracelet
- Each row: border-bottom 1px solid rgba(255,255,255,0.07)
- Use accurate specs for the watch in the video

- CLOSING CTA — `app/components/ClosingCTA.tsx`
 - Full-width, centered, black background
 - Accent label: "Yours to Command"
 - h2 Playfair Display, `clamp(2.2rem, 5vw, 4.5rem)`:
 - "A century of mastery." (regular)
 - "One expression of it." (italic, #E5E5E5)
 - Subtext: Inter 300, #E5E5E5, max 480px
 - CTA button: "Find an Authorised Retailer", same style as hero button but with border 1px solid accent and hover: bg #000, color accent
 - Radial glow behind button:
 - `radial-gradient(ellipse, rgba(200,169,110,0.10) 0%, transparent 70%)`
 - Framer Motion whileHover on button

STEP 7 — WIRE EVERYTHING

`app/page.tsx`:

```
import ScrollHero, FeaturesSection, SpecsSection, ClosingCTA
Render in order inside <main style={{ background: '#000' }}>
```

STEP 8 — RUN

```
npm install && npm run dev
```

Then confirm:

- `http://localhost:3000` loads
- `/frames/frame_0001.jpg` returns 200
- Scrolling through the hero scrubs from assembled watch to full deconstruction, frame by frame
- Features, Specs, and CTA sections animate in on scroll

RULES (non-negotiable)

- No lorem ipsum — all copy fits a luxury watch brand
- No navbar, footer, or cookie banner
- No video element — canvas + JPEG frames only for the scrub
- No scroll event listener — rAF loop with `getBoundingClientRect` only
- Mobile responsive: stack grid to single column below 768px
- No text below #888888 contrast on black
- Do not ask clarifying questions — start with Step 1 immediately

Adapt it to any product: Swap the Rolex details for your client's product. Keep the structure, the step format, and the black background instruction. The prompt works for any luxury item with clean product photography or video.

VI.

HOW TO DELIVER TO CLIENTS

What Claude generates is a complete, deployable Next.js project. Here is how to package and price it:

DELIVERY OPTIONS

- **Standalone handoff:** Zip the project folder and hand it to the client's developer with a Loom walkthrough. Fastest option, zero hosting overhead on your end.
- **Deployed delivery:** Deploy to Vercel in under 2 minutes using the free tier. Hand the client a live URL. This closes higher-ticket deals.
- **Retainer:** Offer to update the product video and copy quarterly as their campaign evolves. Easy recurring revenue.

PRICING ANCHOR

- Scroll-driven product landing page: **\$3,000 to \$6,000**
- Full product campaign site (multiple pages): **\$6,000 to \$12,000**
- Quarterly campaign refresh retainer: **\$1,000 to \$2,000/month**

The client sees a cinematic product experience that agencies charge \$10,000 for. They do not need to know it took one Claude Code session. They are paying for the outcome.

WANT MORE LIKE THIS?

**FOLLOW FOR DAILY
CLAUDE CODE TIPS**

@dak.bugreja

Instagram

Save this guide